UNIVERSITÉ HASSAN 1er – SETTAT
École Nationale des Sciences Appliquées BERRECHID

Département de Mathématique et Informatique
Filière : Ingénierie des Systèmes d'Information et Big Data

Guide d'Installation et d'Exécution

TaxiFlow

Orchestration d'un pipeline MLOps avec Apache Airflow et Docker Compose

Ingestion → Qualité (Great Expectations) → Entraînement MLlib (MLflow)
→ Déploiement → Monitoring



Module : Big Data et Applications

Réalisé par :
Salma Bouziane Ouaritini

Encadré par :
Pr. Lamia Karim

Année Universitaire 2026–2027

Table des matières

1 Introduction **3**
1.1 Configuration requise . . . 3
1.2 Déroulement . . . 3

2 Installation et vérification de Docker Desktop **3**
2.1 Vérification de l'état de Docker . . . 4
2.2 Limitation de la mémoire allouée à Docker . . . 4

3 Préparation de la structure du projet **4**
3.1 Conventions communes à tout le projet . . . 5

4 Les fichiers de configuration Docker **6**
4.1 Le fichier requirements.txt . . . 6
4.2 Le fichier Dockerfile (image Airflow) . . . 6
4.3 Le fichier mlflow/Dockerfile . . . 6
4.4 Le fichier docker-compose.yml . . . 6

5 Le code du pipeline (dossier src) **8**
5.1 spark_utils.py : la session Spark . . . 8
5.2 lineage.py : les jeux de données tracés . . . 8
5.3 ingest.py : extraction et ingestion (zone Bronze) . . . 8
5.4 validate.py : contrôle qualité (Great Expectations) . . . 9
5.5 features.py : préparation des données (zone Silver) . . . 9
5.6 train.py : entraînement (Spark MLlib + MLflow) . . . 10
5.7 deploy.py : déploiement champion / challenger (zone Gold) . . . 10
5.8 monitor.py : monitoring de la dérive (Evidently) . . . 10
5.9 kpis.py : indicateurs pour le tableau de bord . . . 11

6 Le DAG Airflow (dossier dags) **11**

7 Le script de test (dossier scripts) **12**

8 Le tableau de bord (dossier dashboard) **12**
8.1 dashboard/Dockerfile . . . 12
8.2 dashboard/.streamlit/config.toml . . . 12
8.3 dashboard/app.py . . . 12

9 Construction des images **13**

10 Lancement de la plateforme **13**

11 Vérification des conteneurs en cours d'exécution **13**

12 Accès aux interfaces graphiques **14**

13 Exécution du pipeline **14**
13.1 Activation du DAG . . . 14
13.2 Lancement d'un mois . . . 14
13.3 Vérification de chaque tâche . . . 14
13.4 Traitement de plusieurs mois . . . 15
13.5 Vérification des fichiers produits . . . 15

14 Test du contrôle qualité **15**

15 Consultation des résultats **16**
15.1 MLflow 16
15.2 Tableau de bord 16
15.3 Marquez 16

16 Arrêt de la plateforme **16**

17 Problèmes fréquents **16**
17.1 Docker refuse de démarrer : disque plein 16
17.2 L'entraînement est très long ou s'interrompt 17
17.3 Autres problèmes 17

18 Conclusion **17**

1. Introduction

Ce document est un guide pas à pas pour **construire soi-même** le projet **TaxiFlow**, depuis un dossier vide jusqu'aux résultats, sous Windows, avec Docker et Docker Compose. Le guide ne fournit pas le code : pour chaque fichier, il décrit précisément ce que le fichier doit contenir et faire (étapes, paramètres, conventions, résultat attendu). La personne qui réalise la manipulation écrit elle-même le code, puis le vérifie grâce aux contrôles indiqués à chaque étape.

TaxiFlow est un pipeline MLOps qui prédit la durée des courses de taxi à New York à partir des données ouvertes de la NYC Taxi & Limousine Commission (TLC). Le traitement d'un mois de données est orchestré par un DAG **Apache Airflow** qui enchaîne :

— l'**ingestion** des données dans un data lake (zones Bronze, Silver, Gold) ;
— un **contrôle qualité** avec **Great Expectations** ;
— l'**entraînement** de deux modèles **Spark MLlib**, suivis dans **MLflow** ;
— le **déploiement** du meilleur modèle (stratégie champion / challenger) ;
— le **monitoring** de la dérive des données avec **Evidently**.

Les résultats sont affichés dans un tableau de bord **Streamlit** et la traçabilité des données est assurée par **OpenLineage** et **Marquez**.

1.1. Configuration requise

Élément	Configuration
Système	Windows 10 ou 11, 64 bits
Mémoire vive	8 Go minimum (16 Go conseillés)
Espace disque	20 Go libres (les images construites occupent environ 13 Go)
Logiciels	Docker Desktop (avec WSL 2), un éditeur de code (VS Code conseillé), un navigateur web
Connaissances	Python, notions de PySpark, de Docker et d'Airflow
Réseau	Connexion Internet (images Docker et données NYC TLC)

1.2. Déroulement

Étapes	Contenu
2	Installation de Docker Desktop
3 à 8	Écriture du projet : dossiers, fichiers Docker, code du pipeline, DAG, script de test, tableau de bord
9 à 12	Construction des images, lancement, vérification, accès aux interfaces
13 à 15	Exécution du pipeline, test du contrôle qualité, consultation des résultats
16 à 17	Arrêt de la plateforme et résolution des problèmes

2. Installation et vérification de Docker Desktop

Télécharger Docker Desktop pour Windows depuis le site officiel (https://www.docker.com/products/docker-desktop/) et procéder à l'installation en veillant à activer l'option **WSL 2** (Windows Subsystem for Linux). Si WSL n'est pas encore installé, ouvrir PowerShell en administrateur et exécuter la commande suivante, puis redémarrer l'ordinateur :

```
wsl --install
```

2.1. Vérification de l'état de Docker

Lancer Docker Desktop et attendre que l'icône de la baleine, dans la barre des tâches, cesse de s'animer. Ouvrir ensuite PowerShell et exécuter :

```
docker --version
docker compose version
docker run --rm hello-world
```

Les deux premières commandes affichent un numéro de version ; la troisième affiche le message `Hello from Docker!`.

2.2. Limitation de la mémoire allouée à Docker

Sur une machine de 8 Go, il faut limiter la mémoire utilisée par WSL (et donc par Docker) afin de laisser de la place à Windows. Créer le fichier `.wslconfig` dans le dossier de l'utilisateur :

```
notepad "$env:USERPROFILE\.wslconfig"
```

Y insérer le contenu suivant et enregistrer :

```
[wsl2]
memory=5GB
processors=4
swap=2GB
```

Appliquer le réglage en arrêtant WSL, puis relancer Docker Desktop :

```
wsl --shutdown
```

3. Préparation de la structure du projet

Créer le dossier de travail et ses sous-dossiers depuis PowerShell :

```
cd $env:USERPROFILE
mkdir taxiflow
cd taxiflow
New-Item -ItemType Directory -Force -Path dags, src, scripts, mlflow, dashboard\.
    ↪ streamlit, data, reports
code .
```

IMPORTANT : le dossier principal doit s'appeler exactement `taxiflow`. Docker Compose utilise ce nom pour nommer les images ; l'image du tableau de bord pourra ainsi être construite à partir de l'image `taxiflow-airflow`.

Le tableau suivant donne la liste des fichiers à écrire au cours des étapes 4 à 8. Chaque étape décrit précisément ce que le fichier doit faire ; l'écriture du code est laissée à la personne qui réalise la manipulation.

Fichier	Étape	Rôle
`requirements.txt`	4	Librairies Python ajoutées à l'image Airflow
`Dockerfile`	4	Image Airflow + Java + Spark + librairies
`mlflow/Dockerfile`	4	Image du serveur MLflow
`docker-compose.yml`	4	Description des 7 services
`src/spark_utils.py`	5	Création de la session Spark
`src/lineage.py`	5	Déclaration des jeux de données tracés
`src/ingest.py`	5	Extraction + zone Bronze
`src/validate.py`	5	Contrôle qualité (Great Expectations)
`src/features.py`	5	Préparation des données (zone Silver)
`src/train.py`	5	Entraînement (Spark MLlib + MLflow)
`src/deploy.py`	5	Champion / challenger + zone Gold
`src/monitor.py`	5	Dérive des données (Evidently)
`src/kpis.py`	5	Indicateurs du tableau de bord
`dags/taxiflow_pipeline.py`	6	Le DAG Airflow
`scripts/corrupt_data.py`	7	Fabrication d'un mois corrompu
`dashboard/Dockerfile, app.py, .streamlit/config.toml`	8	Tableau de bord Streamlit

3.1. Conventions communes à tout le projet

Pour que les fichiers fonctionnent ensemble, respecter les conventions suivantes.

Élément	Convention
Mois traité	Chaîne au format `AAAA-MM`, passée en paramètre à chaque fonction
Dossier des données	Variable d'environnement `DATA_DIR` (dans le conteneur : `/opt/airflow/data`)
Dossier des rapports	Variable d'environnement `REPORTS_DIR` (dans le conteneur : `/opt/airflow/reports`)
Fichier brut	`data/raw/yellow_tripdata_AAAA-MM.parquet`
Zones du data lake	`data/bronze/yellow/`, `data/silver/yellow/`, `data/gold/predictions/`, avec un sous-dossier par mois : `year=AAAA/month=MM`
Écriture	Mode « overwrite » : relancer un mois remplace ses données au lieu de les dupliquer
Rapports	Chaque étape écrit un fichier JSON `reports/<étape>_AAAA-MM.json` et affiche une ligne de synthèse dans le log
Aléatoire	Graine fixe (`seed = 42`) pour tous les tirages, afin que les résultats soient reproductibles
Cible du modèle	`duration_min` : durée de la course en minutes (arrivée - départ)

4. Les fichiers de configuration Docker

4.1. Le fichier requirements.txt

Créer à la racine un fichier `requirements.txt` qui fixe les versions exactes des librairies suivantes (ces versions ont été testées ensemble avec Airflow 2.11.2 et Python 3.11) :

Librairie	Version	Utilisation
pyspark	3.5.8	Traitement des données et modèles MLlib
mlflow-skinny	3.17.0	Client MLflow (suivi et registre des modèles)
great-expectations	1.24.0	Contrôle qualité
evidently	0.7.23	Détection de la dérive des données

4.2. Le fichier Dockerfile (image Airflow)

Créer à la racine un fichier `Dockerfile` (sans extension) qui réalise, dans l'ordre :

1. partir de l'image officielle `apache/airflow:2.11.2-python3.11` ;
2. passer en utilisateur `root` et installer avec `apt-get` les paquets `default-jre-headless` (Java, indispensable à Spark) et `procps` (commande `ps`, utilisée par Spark), puis nettoyer le cache d'apt ;
3. définir la variable `JAVA_HOME=/usr/lib/jvm/default-java` ;
4. revenir à l'utilisateur `airflow`, copier `requirements.txt` dans l'image et l'installer avec `pip`, en fixant dans la même commande `apache-airflow==${AIRFLOW_VERSION}` pour que pip ne modifie pas la version d'Airflow ;
5. dans une instruction `RUN` séparée, installer `apache-airflow-providers-openlineage==2.11.0` et `cryptography==44.0.3`, toujours en fixant la version d'Airflow.

IMPORTANT : la version `cryptography==44.0.3` est obligatoire : avec une version plus récente, pyOpenSSL provoque une erreur et l'interface web d'Airflow ne répond plus (`ERR_EMPTY_RESPONSE`). Placer OpenLineage dans une couche séparée évite de retélécharger PySpark si on modifie cette ligne.

4.3. Le fichier mlflow/Dockerfile

Dans le dossier `mlflow`, créer un `Dockerfile` qui part de `python:3.11-slim`, installe `mlflow==3.17.0`, désactive la télémétrie (`MLFLOW_DISABLE_TELEMETRY=true`), crée le dossier `/mlflow/artifacts`, se place dans `/mlflow` et expose le port 5000.

4.4. Le fichier docker-compose.yml

Créer à la racine le fichier `docker-compose.yml` décrivant les sept services ci-dessous.

Service	Image / build	Port	Particularités
postgres	postgres:16-alpine	–	Utilisateur, mot de passe et base airflow ; volume pour les données ; test de santé pg_isready
mlflow	build ./mlflow	5000	Commande mlflow server avec une base SQLite /mlflow/mlflow.db, les artefacts dans /mlflow/artifacts, un volume /mlflow, et -allowed-hosts autorisant mlflow et localhost
airflow	build .	8080	Voir le détail ci-dessous
dashboard	build ./dashboard	8501	Nom d'image taxiflow-dashboard ; dossier reports monté en lecture seule ; limite mémoire 400 Mo
marquez-db	postgres:14-alpine	–	Base marquez ; profil lineage
marquez-api	marquezproject/ marquez:0.51.1	5002	Port interne 5000 exposé sur 5002 ; SEARCH_ENABLED=false ; Java limité à 256 Mo ; profil lineage
marquez-web	marquezproject/ marquez-web:0.51.1	3000	Variables MARQUEZ_HOST=marquez-api, MARQUEZ_PORT=5000 ; profil lineage

Le service airflow. C'est le service principal. Il doit :

— s'exécuter avec l'utilisateur 50000:0 et dépendre de postgres (en bonne santé) et de mlflow ;
— monter les dossiers dags, src, scripts, data et reports dans /opt/airflow/ ;
— au démarrage : supprimer un éventuel fichier airflow-webserver.pid, exécuter airflow db migrate, créer l'utilisateur admin / admin avec le rôle Admin, lancer airflow scheduler en arrière-plan puis airflow webserver ;
— recevoir les variables d'environnement du tableau suivant.

Variable	Valeur et rôle
AIRFLOW__CORE__EXECUTOR	LocalExecutor
AIRFLOW__DATABASE__ SQL_ALCHEMY_CONN	Connexion PostgreSQL vers le service postgres
AIRFLOW__CORE__LOAD_EXAMPLES	false (pas de DAG d'exemple)
AIRFLOW__CORE__PARALLELISM	2 ; et MAX_ACTIVE_TASKS_PER_DAG à 1
AIRFLOW__WEBSERVER__WORKERS	1 (économie de mémoire)
PYTHONPATH	/opt/airflow/src (le DAG importe les modules de src)
DATA_DIR / REPORTS_DIR	/opt/airflow/data et /opt/airflow/reports
MLFLOW_TRACKING_URI	http://mlflow:5000
SPARK_DRIVER_MEMORY	1g
TRAIN_SAMPLE_FRACTION	0.15 : part de la zone Silver utilisée pour l'entraînement
MIN_IMPROVEMENT_MINUTES	0.05 : gain minimal de MAE pour remplacer le champion
AIRFLOW__OPENLINEAGE__NAMESPACE	taxiflow
AIRFLOW__OPENLINEAGE__TRANSPORT	Transport http vers http://marquez-api:5000, point d'entrée api/v1/lineage, délai de 5 s et une seule nouvelle tentative (pour ne jamais ralentir le pipeline si Marquez est arrêté)

Tous les services utilisent restart: unless-stopped, et trois volumes nommés sont déclarés : postgres-data, mlflow-data et marquez-data.

Vérification. La commande suivante doit afficher la configuration complète, sans erreur :

```
docker compose --profile lineage config
```

5. Le code du pipeline (dossier src)

Chaque fichier de cette étape correspond à une tâche du pipeline et expose une fonction principale qui prend le mois en paramètre. Le DAG (étape 6) se contente d'appeler ces fonctions dans l'ordre.

5.1. spark_utils.py : la session Spark

Écrire une fonction `get_spark(app_name)` qui renvoie une session Spark :

- en mode local (`local[2]`) ;
- avec la mémoire du driver lue dans `SPARK_DRIVER_MEMORY` ;
- avec `spark.sql.shuffle.partitions` à 8, le fuseau horaire `UTC` et l'interface web de Spark désactivée ;
- en utilisant le même interpréteur Python pour le driver et les workers (`PYSPARK_PYTHON`).

5.2. lineage.py : les jeux de données tracés

Déclarer les jeux de données du projet et écrire une fonction `datasets(*clés)` qui renvoie, pour chaque clé demandée, un objet `Dataset` d'OpenLineage (pour Marquez) et un objet `Dataset` d'Airflow (pour l'onglet Graph).

Clé	Namespace OpenLineage	Nom
`tlc_open_data`	URL de la NYC TLC	`trip-data/yellow_tripdata`
`taxi_zones`	`taxiflow-datalake`	`raw/taxi_zone_lookup`
`bronze / silver / gold`	`taxiflow-datalake`	`bronze/yellow`, `silver/yellow`, `gold/predictions`
`model`	`mlflow`	`registry/taxiflow-duration-model`
`quality / drift / kpis`	`taxiflow-reports`	`quality_report`, `drift_report`, `kpis_dashboard`

Les noms ne contiennent pas le mois : la zone Silver est un seul jeu de données, dont chaque mois est une partition. Si une des deux librairies est absente, la fonction doit simplement ignorer la forme correspondante.

5.3. ingest.py : extraction et ingestion (zone Bronze)

Écrire deux fonctions.

`extract_month(month)` garantit la présence des fichiers bruts :

- si `data/raw/yellow_tripdata_AAAA-MM.parquet` est absent, le télécharger depuis `https://d37ci6vzurychx.cloudfront.net/trip-data/yellow_tripdata_AAAA-MM.parquet` ;
- faire de même avec la table des zones `https://d37ci6vzurychx.cloudfront.net/misc/taxi_zone_lookup.csv` ;
- télécharger d'abord dans un fichier temporaire (`.part`) puis le renommer, pour ne jamais laisser un fichier incomplet ; envoyer un en-tête `User-Agent` et prévoir un délai maximal de 120 s ;
- ne jamais retélécharger un fichier déjà présent.

`ingest_month(month)` range le mois en zone Bronze :

1. lire le fichier brut avec Spark ;
2. garder les 12 colonnes `VendorID`, `tpep_pickup_datetime`, `tpep_dropoff_datetime`, `passenger_count`, `trip_distance`, `RatecodeID`, `store_and_fwd_flag`, `PULocationID`, `DOLocationID`, `payment_type`, `fare_amount`, `total_amount`, en leur imposant un type fixe (pour que tous les mois aient le même schéma) ;
3. conserver 15 % des courses par tirage aléatoire reproductible (variable `SAMPLE_FRACTION`, valeur par défaut 0.15) ;
4. ajouter les colonnes de traçabilité `ingested_at` (date d'ingestion) et `source_file` ;
5. écrire en Parquet dans `data/bronze/yellow/year=AAAA/month=MM` ;
6. écrire `reports/ingestion_AAAA-MM.json` (nombre de lignes brutes et en Bronze) et afficher `[Ingestion] ... courses brutes -> ... en Bronze`.

5.4. validate.py : contrôle qualité (Great Expectations)

Écrire une fonction `validate_month(month)` qui lit la zone Bronze du mois, ajoute la colonne `duration_min`, puis vérifie avec Great Expectations les 18 règles ci-dessous. Le paramètre *mostly* indique la part minimale de lignes qui doivent respecter la règle.

Règle	Tolérance (mostly)
Le lot contient au moins 100 000 courses	–
Les 7 colonnes indispensables existent : heures de départ et d'arrivée, distance, passagers, zones de départ et d'arrivée, prix (une règle par colonne)	–
Heure de départ, zone de départ et zone d'arrivée renseignées (3 règles)	0,99
Départ compris dans le mois traité	0,99
Durée entre 1 min et 3 h	0,95
Distance entre 0,1 et 100 miles	0,95
Entre 1 et 6 passagers	0,85
Prix (`fare_amount`) entre 0 et 1000 $	0,95
Zones de départ et d'arrivée entre 1 et 265 (2 règles)	0,99

La fonction doit ensuite :

— écrire `reports/quality_AAAA-MM.json` (résultat de chaque règle, pourcentage d'erreurs, part de courses valides) et une version HTML lisible ;
— afficher une ligne `[Qualité] OK` ou `[Qualité] KO` par règle, puis `[Qualité] AAAA-MM : x/18 règles OK` ;
— si au moins une règle échoue, **lever une exception** (par exemple `DataQualityError`) : c'est ce qui fait échouer la tâche et arrête le pipeline.

5.5. features.py : préparation des données (zone Silver)

Écrire une fonction `build_features(month)` qui, à partir de la zone Bronze :

1. calcule `duration_min` ;
2. remplace un nombre de passagers vide ou nul par 1, et un `RatecodeID` vide par 99 ;
3. écarte les courses aberrantes, avec les mêmes seuils que le contrôle qualité ;
4. crée les variables : `pickup_hour` (heure), `day_of_week` (jour), `is_weekend`, `is_airport` (départ ou arrivée dans les zones 1, 132 ou 138, ou tarif aéroport), `same_zone` ;
5. ajoute le quartier de départ et d'arrivée (`pickup_borough`, `dropoff_borough`) par jointure avec la table des zones, en remplaçant les valeurs absentes par `Unknown` ;

6. écrit en zone Silver uniquement les 12 variables et la cible ;
7. écrit `reports/features_AAAA-MM.json` et affiche `[Features] ... courses Bronze -> ... en Silver`.

ATTENTION : le prix (`fare_amount`, `total_amount`) ne doit pas faire partie des variables du modèle : il est calculé après la course et contient déjà l'information de durée (fuite de données).

5.6. train.py : entraînement (Spark MLlib + MLflow)

Écrire une fonction `train_month(month)` qui :

1. lit la zone Silver, en garde la part `TRAIN_SAMPLE_FRACTION`, puis la découpe en 80 % entraînement / 20 % test (graine 42) ;
2. construit un pipeline Spark ML : `StringIndexer` pour les deux quartiers, `VectorAssembler` pour réunir toutes les variables, puis le modèle ;
3. entraîne deux modèles : `LinearRegression` (`maxIter=50`, `regParam=0.1`) et `GBTRegressor` (`maxIter=30`, `maxDepth=6`) ;
4. évalue chacun sur le jeu de test avec `RegressionEvaluator` : MAE, RMSE et R^2 ;
5. enregistre chaque modèle dans un run MLflow de l'expérience `taxiflow-duration` : paramètres, métriques, tag `month`, et le modèle lui-même (`mlflow.spark.log_model`) ; pour le GBT, enregistrer aussi l'importance des variables ;
6. inscrit le modèle ayant la plus petite MAE dans le registre MLflow, sous le nom `taxiflow-duration-model` (nouvelle version) ;
7. écrit `reports/training_AAAA-MM.json` et affiche une ligne `[Entraînement] ... MAE = ... min` par modèle.

La fonction renvoie le numéro de la version inscrite : la tâche suivante en a besoin.

5.7. deploy.py : déploiement champion / challenger (zone Gold)

Écrire trois fonctions.

`compare_models(month, version_challenger)`

— récupère la version portant l'alias `champion` dans le registre (aucune au premier mois) ;
— recharge les deux modèles depuis MLflow et les évalue sur **le même** jeu de test (même découpage, même graine) ;
— décide `promote` si aucun champion n'existe ou si la MAE du challenger est inférieure à celle du champion d'au moins `MIN_IMPROVEMENT_MINUTES` ; sinon `keep` ;
— écrit `reports/deployment_AAAA-MM.json` (versions, métriques, décision, raison en clair) et affiche `[Déploiement] ...`.

`promote_model(month, version)` place l'alias `champion` sur la version indiquée.

`score_batch(month)` charge le modèle champion, prédit la durée de toutes les courses Silver du mois et écrit en zone Gold, pour chaque course, la durée réelle, la durée prédite, l'erreur absolue et la version du modèle utilisée.

5.8. monitor.py : monitoring de la dérive (Evidently)

Écrire une fonction `monitor_drift(month, version_reference)` qui :

— ne fait que noter « pas de référence » au premier mois ;
— sinon, retrouve le mois d'entraînement du modèle de référence (tag `month` de son run MLflow) ;

— prélève environ 20 000 courses du mois de référence et du mois traité, sur les variables `trip_distance`, `pickup_hour`, `day_of_week`, `is_airport`, `passenger_count`, les deux quartiers et `duration_min` ;
— produit un rapport Evidently `DataDriftPreset` (enregistré en HTML) et en extrait, pour chaque variable, le score de dérive et le verdict ;
— lève une **alerte** si au moins 25 % des variables ont dérivé ou si la durée elle-même a dérivé ;
— écrit `reports/drift_AAAA-MM.json`, enregistre le résultat dans une expérience MLflow `taxiflow-monitoring` et affiche `[Monitoring] ... variables ont dérivé`.

5.9. kpis.py : indicateurs pour le tableau de bord

Écrire une fonction `build_kpis(month)` qui lit la zone Gold et produit `reports/kpis_AAAA-MM.json` contenant de petits tableaux agrégés : totaux du mois (courses, durée réelle et prédite moyennes, MAE), puis par heure de départ, par jour de la semaine, par quartier de départ, par tranche de distance, la répartition des erreurs et les principaux trajets quartier → quartier. Afficher `[KPIs] ... courses résumées pour le dashboard`.

6. Le DAG Airflow (dossier dags)

Dans `dags`, créer `taxiflow_pipeline.py` qui définit le DAG `taxiflow_pipeline` avec les caractéristiques suivantes :

— pas de planification (`schedule=None`), pas de rattrapage, **une seule exécution à la fois** (`max_active_runs=1`) ;
— un paramètre `month` (`Param` de type chaîne, valeur par défaut `2024-01`, validé par l'expression régulière `^\d{4}-(0[1-9]|1[0-2])$`), saisi dans le formulaire de lancement ;
— onze tâches `PythonOperator` qui appellent les fonctions de l'étape 5 avec `params["month"]`, sauf `decide_deploy` (`BranchPythonOperator`) et `keep_champion` (`EmptyOperator`) ;
— les résultats sont transmis d'une tâche à l'autre par XCom (par exemple la version du challenger produite par `train_model`) ;
— chaque tâche déclare ses `inlets` et `outlets` à l'aide de `datasets(...)` de `lineage.py`.

Tâche	Fonction appelée	Lit → écrit
`ingest_month`	`ingest_month`	TLC, zones → bronze
`validate_raw`	`validate_month`	bronze → quality
`build_features`	`build_features`	bronze, zones → silver
`train_model`	`train_month`	silver → model
`compare_models`	`compare_models` (version issue de `train_model`)	silver, model
`decide_deploy`	renvoie `promote_model` ou `keep_champion` selon la décision	–
`promote_model`	`promote_model`	→ model
`keep_champion`	aucune	–
`score_batch`	`score_batch`	silver, model → gold
`monitor_drift`	`monitor_drift` (version de l'ancien champion)	silver → drift
`publish_kpis`	`build_kpis`	gold → kpis

Ordre des tâches :

```
ingest_month >> validate_raw >> build_features >> train_model >> compare_models >>
    ↪ decide_deploy
```

```
decide_deploy >> [promote_model, keep_champion] >> score_batch >> monitor_drift >>
    ↪ publish_kpis
```

IMPORTANT : comme une seule des deux branches s'exécute, `score_batch` doit utiliser la règle de déclenchement `none_failed_min_one_success` ; avec la règle par défaut, la tâche sautée bloquerait la suite du pipeline.

7. Le script de test (dossier scripts)

Dans `scripts`, créer `corrupt_data.py`, qui fabrique un faux mois `2024-08` pour tester le contrôle qualité :

1. s'assurer que le fichier de juillet 2024 est présent (en appelant `extract_month("2024-07")`) ;
2. lire ses courses avec `pyarrow` et décaler toutes les dates de 31 jours, pour obtenir un mois d'août ;
3. abîmer volontairement les données (tirage aléatoire reproductible) : 25 % de prix négatifs, 20 % de distances nulles, 10 % de courses dont l'arrivée précède le départ ;
4. écrire le résultat dans `data/raw/yellow_tripdata_2024-08.parquet` et afficher le nombre de lignes abîmées pour chaque anomalie.

8. Le tableau de bord (dossier dashboard)

8.1. dashboard/Dockerfile

Partir de l'image `taxiflow-airflow:latest` (qui contient déjà pandas et pyarrow), installer `streamlit==1.39.0` et `plotly==5.24.1`, copier `app.py` et le dossier `.streamlit` dans `/app`, exposer le port 8501 et lancer `streamlit run app.py` sur l'adresse `0.0.0.0`, port 8501. Penser à vider l'`ENTRYPOINT` hérité de l'image Airflow.

8.2. dashboard/.streamlit/config.toml

Définir un thème sombre (section `[theme]` : fond, couleur principale, couleur du texte) et le mode `headless` du serveur.

8.3. dashboard/app.py

L'application lit les fichiers JSON du dossier `REPORTS_DIR` et propose une liste déroulante pour choisir le mois, un bouton **Actualiser** et cinq onglets :

Onglet	Contenu attendu	Rapports lus
Vue d'ensemble	Courses du mois, part de courses valides, MAE, version en production ; courses et durée par heure ; départs par quartier ; historique des exécutions	ingestion, quality, training, kpis
Qualité des données	Taux d'erreur par règle (en rouge si règle non respectée), part de courses valides par mois, détail des 18 règles	quality
Modèles	MAE et R^2 de la régression linéaire et du GBT par mois, décisions de déploiement	training, deployment
Prédictions	Durée réelle et prédite par heure, erreur selon la distance, répartition des écarts	kpis
Dérive des données	Part de variables en dérive par mois, score de chaque variable	drift

Si aucun rapport n'existe encore, l'application doit afficher un message invitant à lancer le DAG, au lieu de planter.

9. Construction des images

Avant de construire, vérifier qu'il reste au moins 20 Go libres sur le disque C : :

```
Get-PSDrive C | Select-Object @{n="Libre (Go)";e={[math]::Round($_.Free/1GB,1)}}
```

La construction se fait une seule fois. Elle télécharge l'image officielle d'Airflow, Java, PySpark et les autres librairies : compter de 15 à 30 minutes selon la connexion. Depuis le dossier `taxiflow`, construire d'abord l'image Airflow, car l'image du tableau de bord est bâtie dessus :

```
cd $env:USERPROFILE\taxiflow
docker compose build airflow
```

Puis construire les autres images :

```
docker compose --profile lineage build
```

REMARQUE : si la construction s'interrompt sur une erreur réseau (`unexpected EOF`, `rpc error`), relancer simplement la même commande : les étapes déjà terminées sont gardées en cache.

10. Lancement de la plateforme

Exécuter la commande suivante pour lancer tous les conteneurs en arrière-plan :

```
docker compose --profile lineage up -d
```

L'option `-profile lineage` démarre aussi les trois services de traçabilité (Marquez). Sans ce profil (`docker compose up -d`), le pipeline fonctionne de la même façon, sans la traçabilité.

11. Vérification des conteneurs en cours d'exécution

```
docker compose ps
```

Les sept conteneurs doivent apparaître avec l'état `Up` : `airflow`, `postgres`, `mlflow`, `dashboard`, `marquez-api`, `marquez-db` et `marquez-web`. Les deux bases de données affichent en plus `healthy`.

Au premier démarrage, Airflow initialise sa base et crée le compte administrateur, ce qui prend une à deux minutes. Pour suivre ce démarrage, puis quitter l'affichage avec `Ctrl + C` :

```
docker compose logs -f airflow
```

12. Accès aux interfaces graphiques

Interface	Adresse	Rôle
Apache Airflow	http://localhost:8080	Lancer et suivre le pipeline
MLflow	http://localhost:5000	Comparer les modèles, voir le modèle en production
Tableau de bord	http://localhost:8501	Indicateurs : qualité, modèles, prédictions, dérive
Marquez	http://localhost:3000	Graphe de traçabilité des données

Se connecter à Airflow avec les identifiants créés au démarrage :

— **Username :** `admin`
— **Password :** `admin`

Tant qu'aucun mois n'a été traité, le tableau de bord indique qu'aucun rapport n'est disponible et MLflow ne contient aucune expérience : c'est normal.

13. Exécution du pipeline

13.1. Activation du DAG

Sur la page **DAGs** d'Airflow, le DAG `taxiflow_pipeline` est en pause. Activer l'interrupteur situé à gauche de son nom, puis cliquer sur le nom pour ouvrir sa page.

13.2. Lancement d'un mois

1. Cliquer sur le bouton **Trigger DAG** (le triangle, en haut à droite).
2. Dans le formulaire, saisir le mois à traiter dans le champ `month`, au format `AAAA-MM`, par exemple `2024-01`.
3. Cliquer sur **Trigger**.
4. Ouvrir l'onglet **Graph** et activer **Auto-refresh** pour suivre l'avancement.

Un mois complet dure de 15 à 25 minutes ; l'entraînement représente plus de la moitié du temps. Le DAG n'exécute qu'un mois à la fois : un mois lancé pendant qu'un autre tourne est mis en attente et démarre ensuite automatiquement.

13.3. Vérification de chaque tâche

Cliquer sur la case d'une tâche dans le Graph, puis sur l'onglet **Logs**. Chaque tâche écrit une ligne de synthèse :

Tâche	Ligne attendue dans le log
`ingest_month`	`[Extraction] Téléchargement de ...` puis `[Ingestion] ... courses brutes -> ... en Bronze`
`validate_raw`	`[Qualité] 2024-01 : 18/18 règles OK, ... courses valides`
`build_features`	`[Features] ... courses Bronze -> ... en Silver`
`train_model`	`[Entraînement] ... MAE = ... min`, une ligne par modèle
`compare_models / decide_deploy`	`[Déploiement] 2024-01 :` suivi de la raison de la décision
`promote_model`	`[Déploiement] Version ... promue 'champion'`
`score_batch`	`[Gold] ... prédictions par le modèle v...`
`monitor_drift`	`[Monitoring] ... variables ont dérivé`
`publish_kpis`	`[KPIs] ... courses résumées pour le dashboard`

Couleurs des cases : vert foncé = réussie, vert clair = en cours, rouge = échec, orange = non lancée à cause d'un échec précédent, rose = sautée par le branchement.

13.4. Traitement de plusieurs mois

Traiter ensuite `2024-02` puis `2024-03`, en attendant la fin de chaque mois. Pour le premier mois, aucun modèle n'est en production : le meilleur modèle est promu directement. Pour les mois suivants, le nouveau modèle est comparé au champion : selon le résultat, `promote_model` ou `keep_champion` s'exécute, l'autre apparaît en rose.

13.5. Vérification des fichiers produits

```
dir data\raw                 # fichiers téléchargés
dir data\bronze\yellow       # zone Bronze : un sous-dossier par mois
dir data\silver\yellow       # zone Silver
dir data\gold\predictions    # zone Gold
dir reports                  # rapports de qualité, entraînement, dérive, KPIs
```

14. Test du contrôle qualité

Fabriquer le mois corrompu `2024-08` :

```
docker compose exec airflow python /opt/airflow/scripts/corrupt_data.py
```

Le script affiche le nombre de courses du fichier créé et le détail des anomalies (prix négatifs 25 %, distances nulles 20 %, arrivée avant départ 10 %). Lancer ensuite le DAG avec `month = 2024-08`. Résultat attendu en moins d'une minute :

- `ingest_month` réussit (vert) : l'ingestion range les données telles quelles ;
- `validate_raw` échoue (rouge) : son log liste les règles `[Qualité] KO ...` et se termine par `DataQualityError: Qualité insuffisante pour 2024-08` ;
- toutes les tâches suivantes passent en orange : aucune n'est lancée.

Le modèle en production n'est pas modifié : dans MLflow, l'alias `champion` pointe toujours sur la même version.

15. Consultation des résultats

15.1. MLflow

Ouvrir `http://localhost:5000`, sélectionner l'expérience `taxiflow-duration`, cocher les deux exécutions d'un même mois puis cliquer sur **Compare** pour comparer leurs métriques (MAE, RMSE, R^2). L'onglet **Models**, modèle `taxiflow-duration-model`, indique la version qui porte l'alias `champion`.

15.2. Tableau de bord

Ouvrir `http://localhost:8501`, cliquer sur **Actualiser** puis choisir le mois dans la liste. Cinq onglets sont disponibles : *Vue d'ensemble*, *Qualité des données*, *Modèles*, *Prédictions* et *Dérive des données*.

15.3. Marquez

Ouvrir `http://localhost:3000`, sélectionner le namespace `taxiflow`, ouvrir **Jobs**, cliquer sur `taxiflow_pipeline.publish_kpis` et activer **Full Graph** : le graphe relie la source NYC TLC aux zones Bronze, Silver et Gold, au modèle et aux rapports. Le job `validate_raw` affiche l'état `FAILED` pour le mois corrompu. Dans Airflow, l'option **Show datasets** de l'onglet Graph affiche ces mêmes jeux de données.

16. Arrêt de la plateforme

```
docker compose --profile lineage stop     # arrête les conteneurs, garde tout
docker compose --profile lineage up -d    # relance les conteneurs
docker compose --profile lineage down     # supprime les conteneurs, garde les donn
    ↪ ées
```

ATTENTION : l'option `down -v` supprime aussi les volumes : l'historique d'Airflow, les modèles MLflow et la traçabilité sont alors effacés. Par ailleurs, les logs des tâches sont stockés dans le conteneur Airflow : après une recréation du conteneur, les logs des anciennes exécutions ne sont plus lisibles (`Could not read served logs`), mais les résultats sont conservés.

17. Problèmes fréquents

17.1. Docker refuse de démarrer : disque plein

Symptôme : toutes les commandes `docker` répondent `Docker Desktop is unable to start` et le disque C : est presque plein.

Cause : le disque virtuel de Docker grossit pendant les traitements mais ne rétrécit jamais de lui-même.

Solution : dans PowerShell ouvert en administrateur, arrêter Docker, vider les fichiers temporaires et compacter le disque virtuel :

```
Get-Process "*docker*" -ErrorAction SilentlyContinue | Stop-Process -Force
wsl --shutdown
Remove-Item "$env:TEMP\*" -Recurse -Force -ErrorAction SilentlyContinue
Clear-RecycleBin -Force -ErrorAction SilentlyContinue
$vhdx = "$env:LOCALAPPDATA\Docker\wsl\disk\docker_data.vhdx"
@"
select vdisk file="$vhdx"
attach vdisk readonly
compact vdisk
```

```
detach vdisk
"@ | Set-Content "$env:TEMP\compact.txt"
diskpart /s "$env:TEMP\compact.txt"
```

Relancer ensuite Docker Desktop. Une fois Docker redémarré, supprimer les images inutilisées et le cache de construction, puis refaire le compactage pour récupérer cette place :

```
docker image prune -f
docker builder prune -af
```

17.2. L'entraînement est très long ou s'interrompt

Réduire encore la part de données utilisée pour l'entraînement dans `docker-compose.yml` (par exemple `TRAIN_SAMPLE_FRACTION: "0.1"`), puis recréer le conteneur Airflow :

```
docker compose --profile lineage up -d airflow
```

Dans Airflow, relancer l'exécution à partir de l'entraînement, sans refaire l'ingestion : cliquer sur la case `train_model`, puis **Clear**, cocher **Downstream** et confirmer. Pendant l'entraînement, fermer les applications gourmandes.

17.3. Autres problèmes

Problème	Solution
L'image du tableau de bord ne se construit pas	Construire d'abord `docker compose build airflow` et vérifier que le dossier s'appelle `taxiflow`
`localhost:8080` ne répond pas	Attendre une à deux minutes et consulter `docker compose logs airflow`
Le DAG n'apparaît pas dans Airflow	Erreur dans `taxiflow_pipeline.py` : l'erreur est affichée en haut de la page **DAGs** (vérifier l'indentation)
Une tâche échoue avec `ModuleNotFoundError`	Un fichier du dossier `src` manque ou est mal nommé, ou `PYTHONPATH` n'est pas défini
Une tâche échoue sur une erreur Python	Lire la fin du log de la tâche : le fichier et la ligne en cause y sont indiqués ; corriger puis relancer la tâche avec **Clear**
`score_batch` ne démarre jamais	Règle `none_failed_min_one_success` manquante sur cette tâche
`ingest_month` échoue au téléchargement	Vérifier la connexion Internet, ou placer le fichier du mois à la main dans `data/raw`
Le tableau de bord indique « Aucun rapport »	Aucun mois n'a encore été traité jusqu'au bout ; cliquer sur **Actualiser**
Le tableau de bord affiche `SyntaxError`	Erreur dans `dashboard/app.py` : corriger la ligne indiquée puis recharger la page
Marquez n'affiche aucun job	Vérifier que la plateforme a été lancée avec `-profile lineage`

18. Conclusion

Ce guide a permis de construire soi-même la plateforme TaxiFlow à partir d'un dossier vide et d'exécuter son pipeline MLOps de bout en bout. Les étapes essentielles sont :

— installer Docker Desktop avec WSL 2 et limiter la mémoire allouée ;
— écrire les fichiers du projet en suivant leurs spécifications : configuration Docker, code du pipeline, DAG, script de test et tableau de bord ;

— construire les images, en commençant par l'image Airflow, puis lancer les sept services ;
— déclencher le DAG `taxiflow_pipeline` en choisissant le mois à traiter ;
— vérifier le blocage d'un mois corrompu par le contrôle qualité ;
— consulter les résultats dans MLflow, le tableau de bord et Marquez.

Cette approche par conteneurs garantit un environnement isolé et reproductible : le projet se construit et se lance de la même façon sur n'importe quelle machine équipée de Docker.